

How the Internet Delivers a Web Page

A Case Study of <https://vjcet.org/> (Viswajyothi College of Engineering and Technology)

Student Name:	Eabel Biju John
Roll No:	27
Class:	CSE A
Course:	PCCST501 – Computer Networks
Module:	1
Website chosen :	https://vjcet.org/ (Viswajyothi College of Engineering and Technology)

1. Introduction

Every time a student types <https://vjcet.org/> into a browser and presses Enter, a chain of well-defined, layered protocols works together in a fraction of a second to fetch and display the VJCET college home page. This report traces that journey step by step, using the TCP/IP protocol suite as the backbone of the explanation, and answers the nine questions set out in the assignment.

2. The TCP/IP Protocol Stack

The TCP/IP suite organizes the task of communication into five cooperating layers. Each layer receives services from the layer below it and provides services to the layer above it, so the complex job of moving a web page from a server in the VJCET data centre (or its hosting provider) to a student's browser is broken into smaller, manageable pieces.

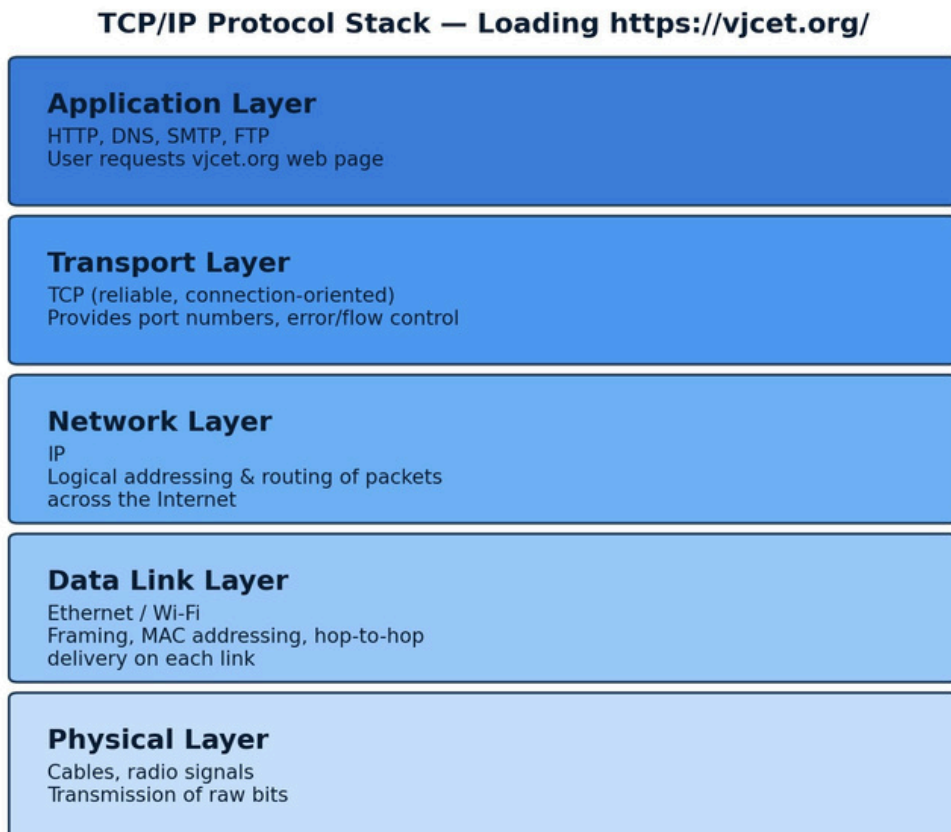


Figure 1: The five-layer TCP/IP protocol stack, applied to a request for vjcet.org

Role of each layer

- Application Layer – Where the user-facing protocols live: HTTP (fetching the vjcet.org page), DNS (resolving the domain name), and possibly SMTP (if the site sends contact-form e-mail). This layer creates the actual request message the student's action triggers.
- Transport Layer – TCP takes the HTTP message and breaks it into segments, adds source/destination port numbers (port 443 for HTTPS), and guarantees reliable, in-order, error-checked delivery between the browser and the VJCET web server.
- Network Layer – IP attaches the logical (IP) addresses of the student's device and the VJCET server to each packet (datagram) and routers along the path use these addresses to forward the packet toward its destination.
- Data Link Layer – Handles hop-to-hop delivery across each physical link (the college/home Wi-Fi or Ethernet, then each ISP link) using MAC addresses and framing, e.g. Ethernet or Wi-Fi (802.11).
- Physical Layer – Converts the frames into actual electrical, optical, or radio signals that travel over cables, fibre, or air between the student's device and the VJCET server.

3. Role of DNS in Converting the Domain Name to an IP Address

Computers route traffic using numeric IP addresses, not names, so the browser must first translate "vjcet.org" into an IP address before any web page can be requested. This is the job of the Domain Name System (DNS), a distributed, hierarchical directory service.

- The browser hands the name vjcet.org to a DNS resolver (client) on the student's machine.
- The resolver queries the local/ISP DNS server; if that server does not already have the mapping cached, it queries a root server, then the .org top-level-domain server, and finally the authoritative name server for vjcet.org.
- The authoritative server returns the IP address bound to vjcet.org (an 'A' or 'AAAA' resource record) back up the chain (recursive resolution) or directly to the resolver (iterative resolution).
- The resolver caches the result for the duration of its Time-to-Live (TTL) so that repeated visits to the site are resolved instantly without repeating the whole lookup.

Only after this name-to-address mapping is complete can the browser open a TCP connection to the actual VJCET web server.

4. How HTTP Requests and Receives the Web Page

Once the browser knows the server's IP address, it uses the HyperText Transfer Protocol (HTTP, almost always over HTTPS/TLS for vjcet.org today) to actually fetch the page:

- The browser's HTTP client composes a request message with a request line — GET / HTTP/1.1 — followed by header lines (Host: vjcet.org, Accept, User-Agent, etc.).
- This request is handed to TCP, which delivers it reliably to the VJCET web server listening on port 443 (HTTPS) or 80 (HTTP).
- The server processes the request and returns a response message consisting of a status line (e.g. HTTP/1.1 200 OK), response headers (Content-Type, Content-Length, Set-Cookie, etc.), and the body — the actual HTML of the home page.
- The browser parses the HTML and discovers further embedded objects (the VJCET logo, CSS stylesheets, JavaScript files, images of the campus, etc.), each of which triggers additional GET requests, typically reused over the same persistent HTTP/1.1 connection to avoid the overhead of opening a new TCP connection for every object.
- Once all objects are received, the browser renders the complete VJCET home page on the screen.

5. How TCP Provides Reliable Communication for HTTP

HTTP itself has no mechanism to guarantee delivery — that reliability is entirely delegated to TCP, the transport-layer protocol beneath it. TCP is connection-oriented and reliable, and provides the following guarantees while carrying the HTTP request and the VJCET server's response:

- Connection establishment (three-way handshake) — SYN, SYN-ACK, ACK — sets up a logical pipe between the browser and the VJCET server before any HTTP data flows.
- Segmentation and sequencing — the HTML page and its objects are broken into segments, each numbered so the receiving side can reassemble them in the correct order even if they arrive out of sequence.
- Error control — checksums detect corrupted segments; missing or damaged segments are automatically retransmitted, so the browser never has to display a corrupted or partial page.
- Flow control — TCP matches the sending rate of the server to what the student's device and network connection can actually handle, preventing buffer overflow.
- Congestion control — TCP backs off its sending rate when the network path is congested, protecting the shared Internet infrastructure.
- Graceful connection termination — once the page and its assets are fully delivered, the TCP connection is closed (or kept alive briefly for further requests) in an orderly way.

6. Client–Server or Peer-to-Peer Architecture?

Accessing <https://vjcet.org/> clearly follows the Client–Server paradigm, not Peer-to-Peer. Justification:

- A dedicated server process (the VJCET web server, e.g. Apache/Nginx) runs continuously and waits for requests — it does not wait for a specific peer, and it does not itself request pages from the visiting browser.
- The student's browser is the client: it is started only when needed, initiates the request, receives the response, and then may close — it never behaves as a server for other students.
- All communication is asymmetric and centralized around the server: many students (clients) connect to the same VJCET server; the server does not distribute the load of hosting the site to the visiting browsers, unlike a P2P file-sharing swarm where every peer both uploads and downloads.
- The service (the website) is available only because the server is guaranteed to be running at a fixed, known address (resolved via DNS); this fixed always-on-server requirement is the hallmark of client-server systems, as opposed to P2P systems such as BitTorrent, where peers come and go and each one can both provide and consume the resource.

7. HTTP vs. FTP — Key Differences

Aspect	HTTP	FTP
Purpose	Retrieves and renders web pages and embedded objects (used to browse vjcet.org)	Transfers files between a client and server, optimized for bulk file uploads/downloads
Connections used	Normally one connection that carries both control information (headers) and data together	Two separate connections: a control connection (port 21) that stays open for the session, and a data connection (port 20) opened per file transfer
Statefulness	Originally stateless; state (e.g. logged-in student) is added artificially using cookies	Inherently stateful — the control connection maintains login state and current directory for the whole session
Typical use on vjcet.org	Delivers the HTML page, CSS, JavaScript, and images that make up the site	Would be used by the college's web administrators to upload new pages, notices, or files to the server, not by an ordinary visitor

8. How Cookies Improve User Experience

The Web was originally designed to be stateless — each HTTP request/response pair is independent, so by default a server like vjcet.org's cannot tell whether two requests came from the same visitor. Cookies solve this: the server sends a small piece of data (a Set-Cookie header) that the browser stores and automatically resends with every later request to the same site, letting the server recognize returning visitors.

Real-world example

When a student logs into the VJCET student/exam portal (or any similar college login page hosted on the site) with a roll number and password, the server creates a session and returns a session-ID cookie. On every subsequent click — checking attendance, downloading a circular, viewing results — the browser automatically includes that cookie, so the server instantly recognises the student without asking for the password again on every page. This makes browsing feel continuous and personalized rather than forcing the student to re-authenticate for every single click, exactly the benefit cookies were designed to provide.

9. Flow Diagram: Browser to Web Server and Back

Flow of Communication: Browser → DNS → VJCET Web Server → Browser

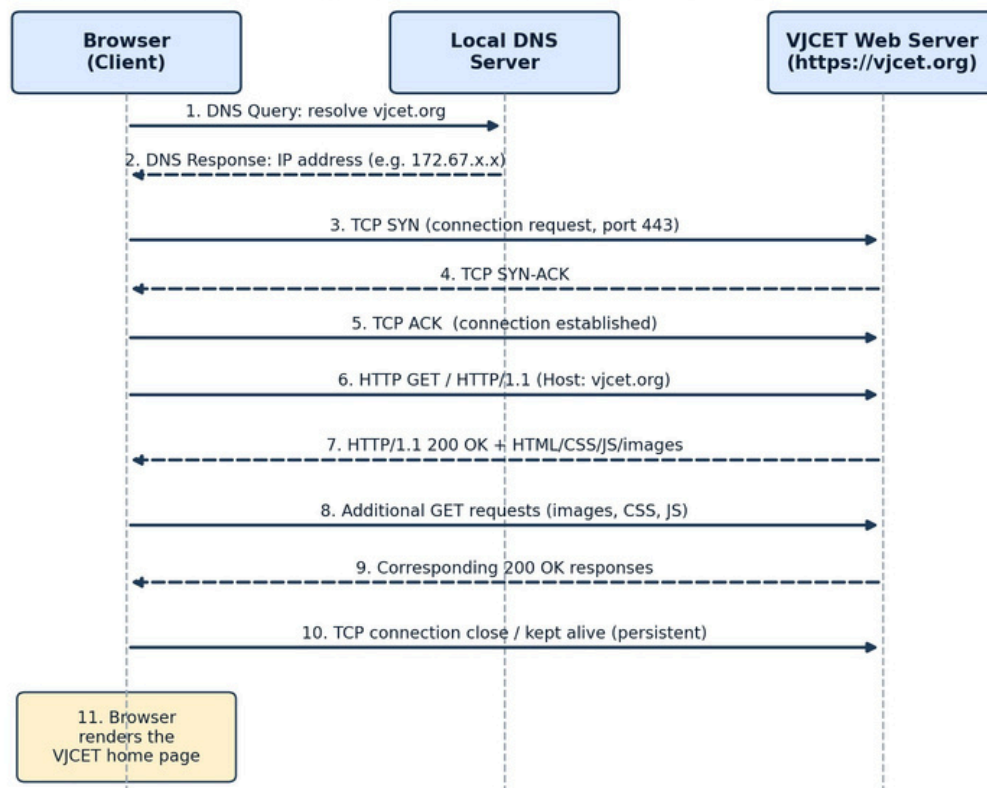


Figure 2: Complete sequence of communication when loading <https://vjcet.org/>

10. Conclusion

Loading a single page such as <https://vjcet.org/> is the visible result of an invisible, tightly coordinated stack of application-layer protocols working over reliable transport and routed delivery. DNS turns a memorable name into a usable address, HTTP defines the language of the request and response, and TCP guarantees that the college's HTML, images, and scripts arrive complete and in order, while cookies add the continuity needed for logins and personalization. Without these standardized, layered application-layer protocols, every website — including a college's own — would need its own custom, incompatible way of exchanging information, making the modern Internet unworkable. Understanding this everyday sequence shows why application-layer protocols, though invisible to most users, are the true foundation of how we search, learn, and communicate online every single day.